

IMAGE PROCESSING

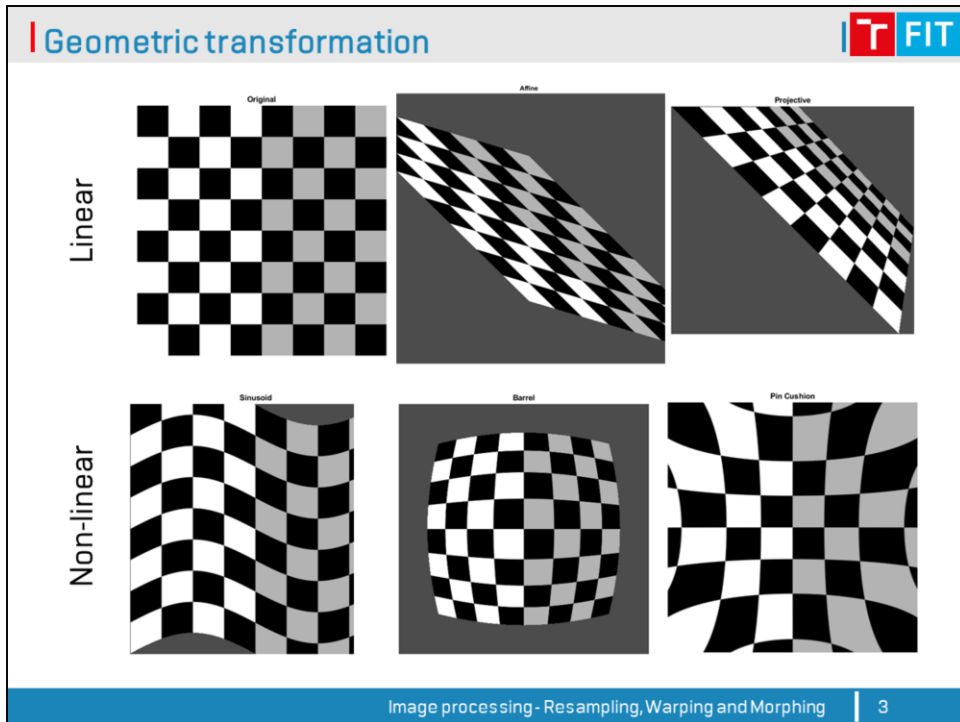
RESAMPLING & WARPING (MORPHING)

Vítá Beran



Brno University of Technology
Faculty of Information Technology
Božetěchova 1/2, 602 00 Brno

- How to change the image size?
- How to bend the image?
- Why resampling is important and how to do it?
- How to do warping and morphing of the image?
- What is an aliasing and how to handle it?



One of the **basic image processing operations**.

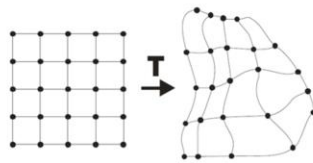
- Linear: displacement, rotation, scaling, skewing, perspective projection
 - scale change, translation, rotation, skew of 2D image (i.e. by known transformation)
- Non-linear: Warping / Morphing, camera/display distortion
 - removing camera geometric distortions (geometric transformation is often computed from the image)

- Geometric transformation of an image is a function:

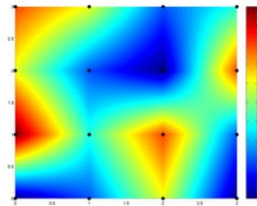
$$T(u, v) = [f_x(u, v), f_y(u, v)]$$

- where inputs of the functions $f_x(u, v)$ and $f_y(u, v)$ are pixel coordinates u and v , and
- the functions f_x , resp. f_y return new positions of the pixel.

1. Mapping



2. Reconstruction

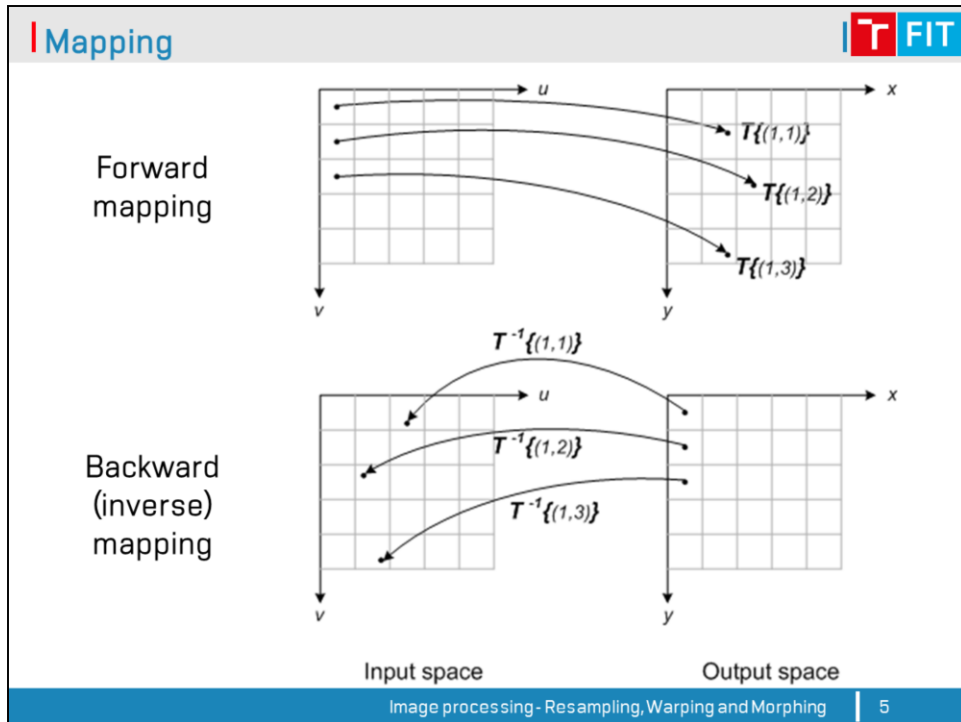


Geometric operations map the pixel (point in the image)

- from input position - coordinates $[u, v]$ (input image)
- to the output coordinates – to the new position - coordinates $[x, y]$ (output image)

The **image transformation** consists of **two steps**:

- Mapping - transformation of pixel coordinates**
new coordinates it may not fit exactly with the image grid, so it is necessary work with real values,
- Reconstruction - calculating new pixel value**
resampling interpolation of surrounding points.



Straightforward approach leads to **Forward mapping**:

1. Iterate over pixels on the input image
2. Maps the coordinates to the output image
3. Set the output pixel value to the input pixel value

But this actually **does not work**. Something bad on the output image – **holes**!

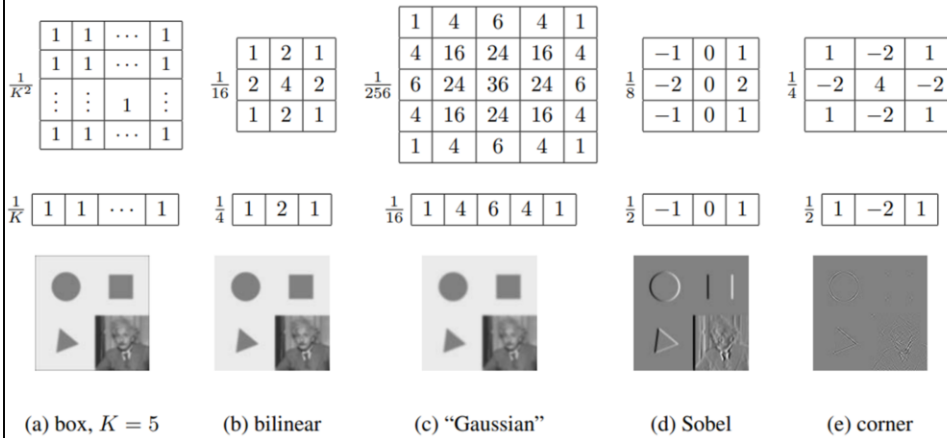
Why?

Function T actually maps the integer number to non-integer number.

The **Backward (inverse) mapping** is a solution to that problem.

1. Compute inverse transformation
2. Iterate over pixels on the output image and maps the coordinates to input image
3. Compute the new intensity value of the output pixel by reconstruction of a non-existing value

$$T(u, v) = F(u, G(v))$$



Szeliski R.: Computer Vision: Algorithms and Applications (pp. 102)

A **separable transformation** is such transformation that can be computed as composition of two transformations separately, each of which is a single variable function only.

Exaples: ... d) Sobel is composed from 2 different 1D filters (derivative in x-coord. (d) and bilinear (c))

Other examples: a two-pass rotation, warping algorithm, Gaussian blur etc.

- Usually polynomial approximation

$$x = \sum_{r=0}^m \sum_{k=0}^{m-r} a_{r,k} u^r v^k$$

$$y = \sum_{r=0}^m \sum_{k=0}^{m-r} b_{r,k} u^r v^k$$

- that is linear with respect to coefficients $a_{r,k}$ and $b_{r,k}$.
- Examples
 - Bilinear transformation

- Bilinear transformation

$$x = a_0 + a_1u + a_2v + a_3uv$$

$$y = b_0 + b_1u + b_2v + b_3uv$$

- Affine transformation

$$x = a_0 + a_1u + a_2v$$

$$y = b_0 + b_1u + b_2v$$

$$\begin{bmatrix} X \\ Y \\ W \end{bmatrix} = A \cdot \begin{bmatrix} U \\ V \\ W \end{bmatrix} \text{ where } A = \begin{bmatrix} a_1 & a_2 & a_0 \\ b_1 & b_2 & b_0 \\ 0 & 0 & 1 \end{bmatrix}$$

Bilinear transformation

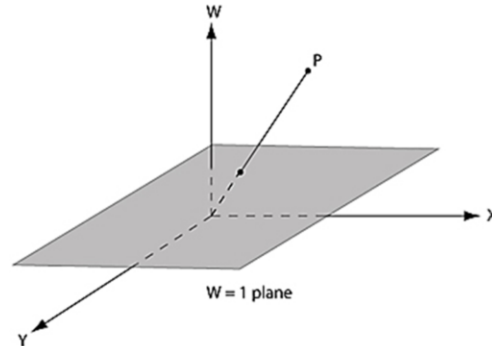
- frequently used in practice
- only 4 pairs are required to calculate transformation parameters corresponding points

Affine transformation

- even simpler than bilinear transformation
- only 3 pairs are required to calculate transformation parameters corresponding points
- Basic geometric transformations such as shift, rotation, scaling or skewing are just affine transformations.
- Using homogeneous coordinates we can represent the affine transformation as matrix A .

- Point $[x, y]$ in Cartesian coordinates is represented in homogeneous coordinates by the vector $[X, Y, W]$ and holds:

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} X/W \\ Y/W \end{bmatrix}$$



- where $W = 1$ for representation of points in 2D Euclidean space and $W = 0$ for vectors.

They are commonly used in projective geometry, computer graphics, robotics, and so on.

They allow to express basic linear transformations or perspective projections using a single matrix.

$$\bullet \mathbf{J} = \begin{bmatrix} \frac{\partial(f_x, f_y)}{\partial(x, y)} \end{bmatrix} = \begin{bmatrix} \frac{\partial f_x}{\partial x} & \frac{\partial f_x}{\partial y} \\ \frac{\partial f_y}{\partial x} & \frac{\partial f_y}{\partial y} \end{bmatrix}$$

- Example: polar-Cartesian transformation

- $x = r \cos \theta$
- $y = r \sin \theta$

$$\bullet \mathbf{J}(r, \theta) = \begin{bmatrix} \frac{\partial x}{\partial r} & \frac{\partial x}{\partial \theta} \\ \frac{\partial y}{\partial r} & \frac{\partial y}{\partial \theta} \end{bmatrix} = \begin{bmatrix} \cos \theta & -r \sin \theta \\ \sin \theta & r \cos \theta \end{bmatrix}$$

- $|\mathbf{J}| = r \cos^2 \theta - r \sin^2 \theta = r$

Geometric transformations applied to the whole image can change the coordinate system.

Determinant of Jacobian matrix $\mathbf{J}$ expresses this change:

- $J = 0$, if the transformation is singular.
- $J = 1$, if an area of the image is not dependent to the transformation.

- Performs a change in the position of each of the points for that section in the x-axis and in the y-axis.

$$\begin{aligned}x &= u + t_u \\ y &= v + t_v \\ J &= 1\end{aligned}$$

$$T(t_u, t_v) = \begin{bmatrix} 1 & 0 & t_u \\ 0 & 1 & t_v \\ 0 & 0 & 1 \end{bmatrix}$$



- Multiply the coordinate of each point with a scaling factor in the x and y axis

$$\begin{aligned}x &= u s_u \\y &= v s_v \\J &= s_u \ s_v\end{aligned}$$

$$S(s_u, s_v) = \begin{bmatrix} s_u & 0 & 0 \\ 0 & s_v & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



- When skewing, multiply the coordinate of each point by the x-y and y-axis skew factor

$$\begin{aligned}x &= u + v.sh_u \\ y &= u.sh_v + v \\ J &= 1\end{aligned}$$

$$Sh(sh_u, sh_v) = \begin{bmatrix} 1 & sh_u & 0 \\ sh_v & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



- More complex transformations (distortion) can be realized by dividing the image into parts and finding each part a simple geometric transformation that is then applies to all parts separately.
- Typical geometric distortions may be
 - by the optical system,
 - non-linearity when scanning line by line or
 - non-constant sampling interval.

- Transformation maps to non-integer coordinates
 - Values on this coordinates are not defined.
1. Find a continuous approximation of a discrete image
 2. Transform it in continuous domain
 3. Sample it back to discrete domain
 4. Use convolution

$$f(x, y) = \sum_{i, j} I(i, j) \cdot h(x - i, y - j)$$

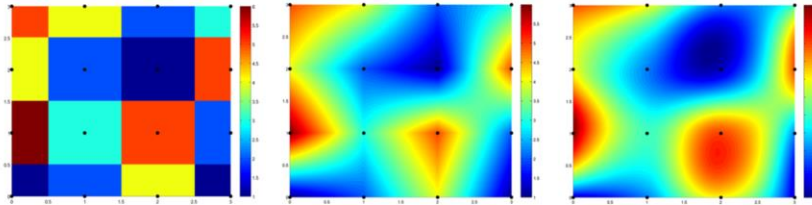
Position of points after transformation coordinate calculation usually disagree with a discrete image raster.

- The brightness value of a point is obtained by luminance interpolation of the surrounding samples.

- The problem of luminance interpolation is usually expressed as a determination

the brightness of a point in the original image that corresponds to the position point of the output image in the grid.

- Nearest neighbor method
- Bilinear interpolation
- Bicubic interpolation



Use to obtain an image value value beyond measured discrete values.

Commonly used methods:

Nearest neighbor method

Bilinear interpolation

loss of sharpness

Bicubic interpolation

achieves better sharpness than bilinear,
color artifacts may arise

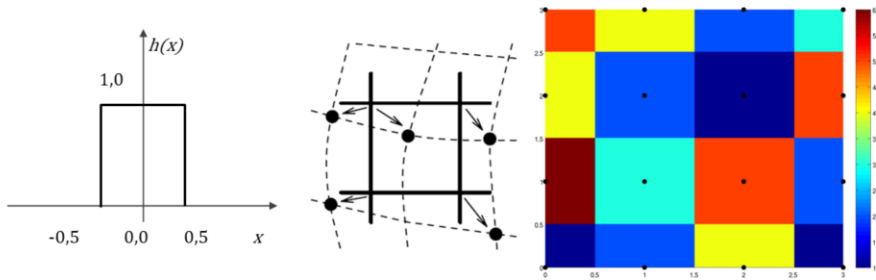
Spline interpolation

In the case of a color image, we perform interpolation for all three color components separately.

$$f(x) = I_i \quad \text{for} \quad i - \frac{1}{2} \leq x < i + \frac{1}{2}$$

- Convolution kernel

$$h(x) = \begin{cases} 1 & \text{for } 0 \leq |x| < \frac{1}{2} \\ 0 & \text{for } |x| \geq \frac{1}{2} \end{cases}$$



Assigns the point (x, y) to the brightness value of the closest point in the discrete grid.

The advantage is speed and color stability.

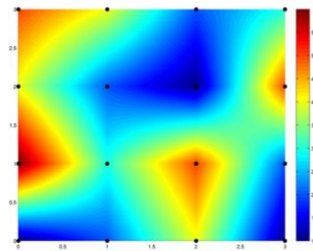
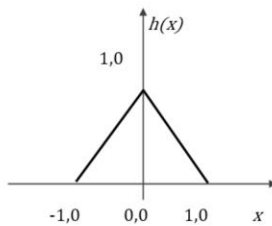
The error in the point position when interpolating the nearest neighbor is up to half a pixel.

This error is noticeable on objects with straight edges, where the edges after the transformation have a stepped character.

$$f(x) = I_i + \frac{x-i}{\Delta x} (I_{i+1} - I_i) \quad \text{for} \quad i: i < x \leq i+1$$

- Convolution kernel

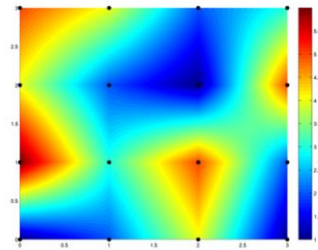
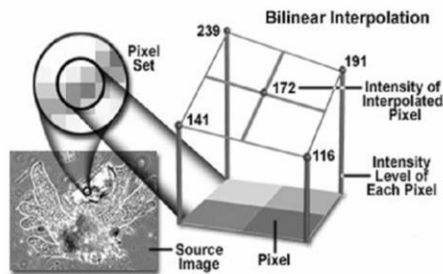
$$h(x) = \begin{cases} 1 - |x| & \text{for } 0 \leq |x| < 1 \\ 0 & \text{for } 1 \leq |x| \end{cases}$$



Let's have two neighbour samples in i and $i+1$ with values $I(i)$ and $I(i+1)$ and look the value of $f(x)$ in point x , where $i < x \leq i+1$.

$$f(x, y) = (1 - a)(1 - b)I_{i,j} + a(1 - b)I_{i+1,j} + b(1 - a)I_{i,j+1} + abI_{i+1,j+1}$$

for $a = x - i: i < x \leq i + 1$
 $b = y - j: j < y \leq j + 1$



Now we have 4 pixels in the neighborhood (x, y) , so we repeat the linear interpolation in both directions.

The influence of each point decreases with the distance.

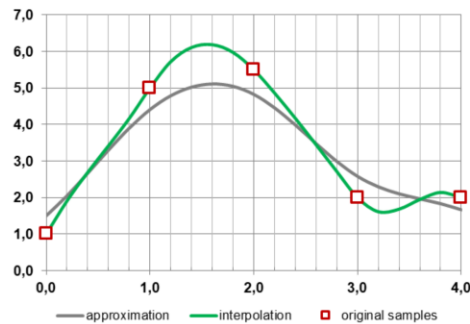
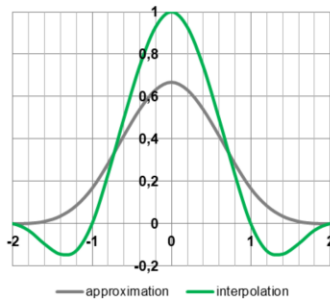
Due to averaging, bilinear interpolation can cause small resolution reduction and blur.

Staircase character problem of straight edges does not occur.

- Interpolation (piecewise-cubic spline)

$$h(x) = \begin{cases} (a+2)|x|^3 - (a+3)|x|^2 + 1 & \text{for } 0 \leq |x| < 1 \\ a(|x|-1)(|x|-2)^2 & \text{for } 1 \leq |x| < 2 \\ 0 & \text{for } 2 \leq |x| \end{cases}$$

- for $a = -1 \dots$



Based on the local interpolation of the pixelated function by a 4-point polynomial.

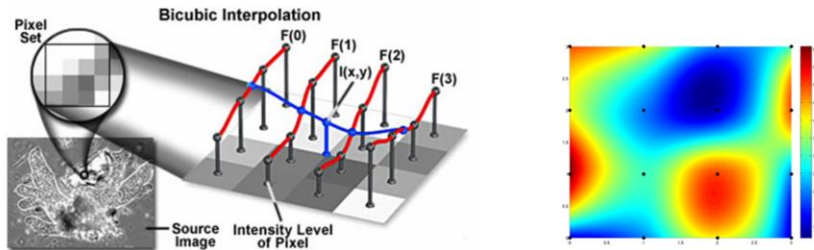
Cubic B-spline curve is often used, where a specifies the derivative at $x=1$ (a is often set to -1).

Most photo editing packages use this *cubic interpolant*, that is C^1 (derivative-continuous) piecewise-cubic *spline*.

The approximation example shown to compare the interpolation and approximation difference.

$$h(x) = \begin{cases} |x|^3 - 2|x|^2 + 1 & 0 \leq |x| < 1 \\ -|x|^3 + 5|x|^2 - 8|x| + 4 & 1 \leq |x| < 2 \\ 0 & 2 \leq |x| \end{cases}$$

$$f(x) = \sum_i I(i) \cdot h(x - i)$$



Based on the local interpolation of the pixelated function by a 16-point polynomial.

Compute in 2 steps:

1. Compute new 4 values in one dimension (e.g. x),
2. Compute final value using these 4 new values in second dimension (e.g. y)

The „staircase“ problem or smoothing at cubic interpolation does not occur, and good image resolution is preserved.

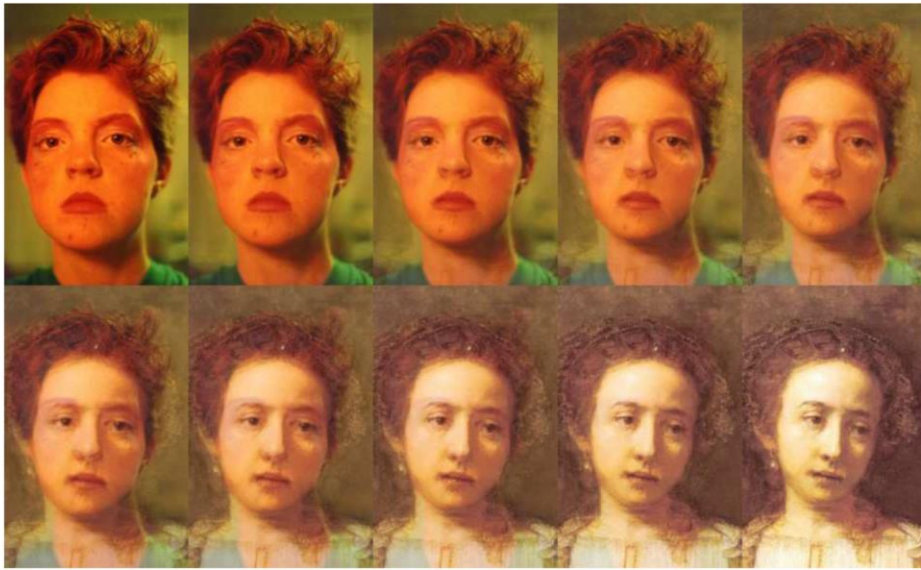
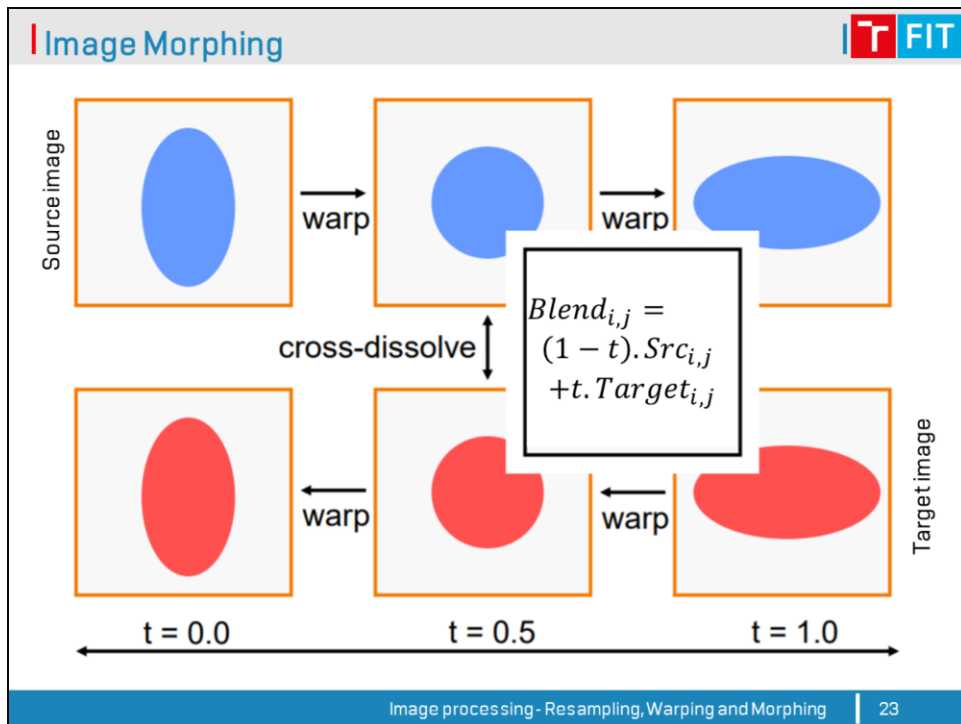


Image Morphing is an image processing technique to turn one image into another through a smooth transition.

Source image is where the morphing starts.

Target Image is where the morphing ends.

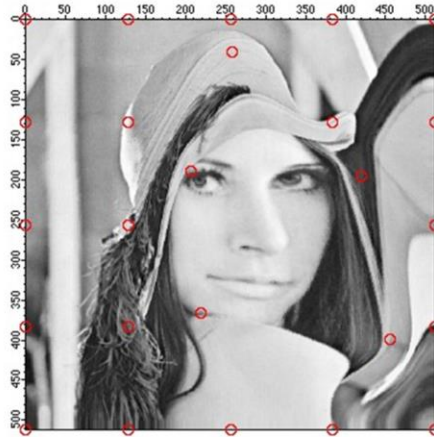
Intermediate frames are the morphed images.



Sequence of:

1. warping from interpolated control structures and
2. cross-dissolving to calculate the resulting image.

- Mesh warping
- Line (feature based) warping
- Display/lens distortions

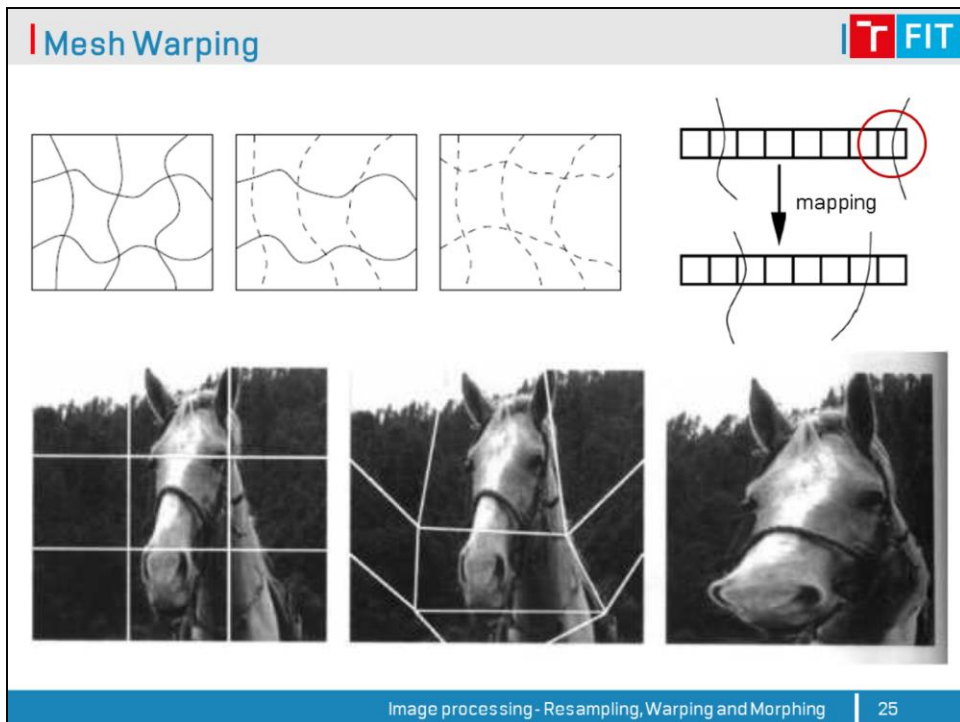


Warping is an application of nonlinear spatial transformation to *one image*.

Depending on how the control structure is entered, we can warping split into

- Mesh warping and
- Line warping.

Warping also includes **distortion of display devices (or camera lens)**.



Mesh warping

The control structure is a defined net of control points that is laid on picture.

The grid curves may be

- polyline,
- Bezier cubes,
- B-spline curves, etc.

Edit the curve to determine its new position and thus the transformation image.

The actual algorithm is two-pass and separable (lines and then columns).

Each pass consists of two steps:

1. The intersection of the vertical curves with the lines of the image is determined first.

2. The defined intervals are interpolated (usually linearly).

The problem is at the end points of the interval that contribute only by reduced intensity (by the percentage of their coverage in the interval).

The second pass is similar, just applied to horizontal curves and columns.

One of the drawbacks of network warping is that **some intuitive operations are difficult to realize** (e.g. changing the position of the corners of the mouth).

Line (Feature based) Warping
T
FIT

$$u = \frac{(X - P) \cdot (Q - P)}{\|Q - P\|^2}$$

$$v = \frac{(X - P) \cdot (Q - P)_{\text{perp.}}}{\|Q - P\|}$$

$$vX' = P' + u(Q' - P') + \frac{v(Q' - P')_{\text{perp.}}}{\|Q' - P'\|}$$

Image processing - Resampling, Warping and Morphing
26

Feature based warping is useful for local changes/warping.

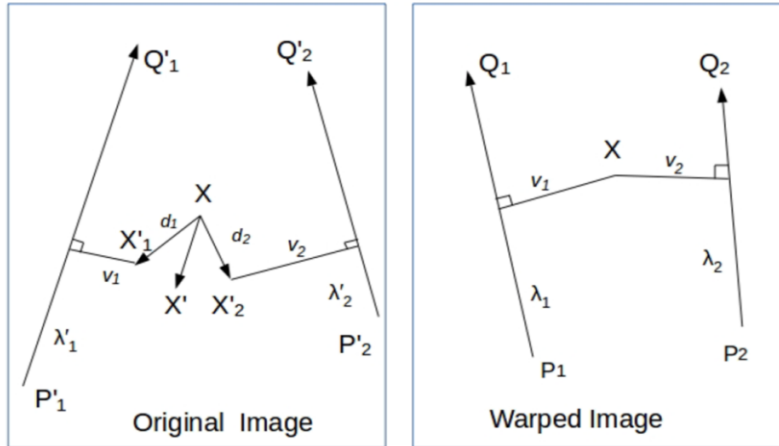
We have 2 control lines segments (features) – in source (P', Q') and destination (P, Q) image.

First, we compute the relative position (u, v) of point X to line segment (P, Q) .

We use these relative mapping to find the position of X' , relative to line segment (P', Q') .

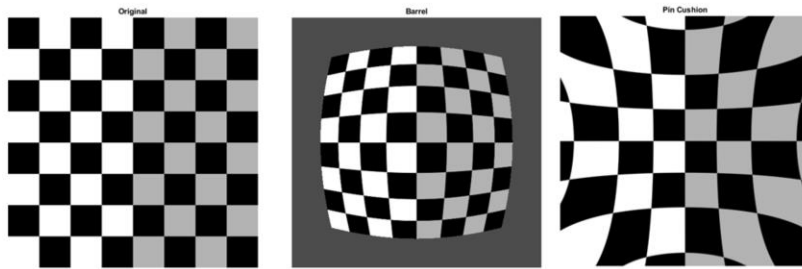
See the translation, rotation and scale in the equation.

$$weight = \left(\frac{length^p}{a + distance} \right)^b$$



When more feature line segments are used for warping:

1. Each feature line segment is associated with a **weight** determining the influence.
Parameters a , b , and p control the influence of distance, weight, and length.
2. The position of X' is computed as the **weighted mean** of projection of X by every feature line segment.



$$\begin{aligned}x' &= a_0 + a_1x + a_2y + a_3x^2 + a_4xy + a_5y^2 \\y' &= b_0 + b_1x + b_2y + b_3x^2 + b_4xy + b_5y^2\end{aligned}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a_0 & \dots & a_5 \\ b_0 & \dots & b_5 \end{bmatrix} \begin{bmatrix} 1 & x & y & x^2 & xy & y^2 \end{bmatrix}^T$$

Spatial distortions caused by display/lens device can be compensated by mapping of the the second, or possibly higher order polynomial.

Example of the polynomial defirmation of the second order (and matrix notation).

$$E = (\mathbf{x}' - \mathbf{A}\mathbf{a})^T(\mathbf{x}' - \mathbf{A}\mathbf{a}) + (\mathbf{y}' - \mathbf{A}\mathbf{b})^T(\mathbf{y}' - \mathbf{A}\mathbf{b})$$

$$\mathbf{A} = \begin{bmatrix} 1 & x_1 & y_1 & x_1^2 & x_1 y_1 & y_1^2 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_M & y_M & x_M^2 & x_M y_M & y_M^2 \end{bmatrix}$$

$$\mathbf{a} = \mathbf{A}^{-}\mathbf{x}'$$

$$\mathbf{b} = \mathbf{A}^{-}\mathbf{y}'$$

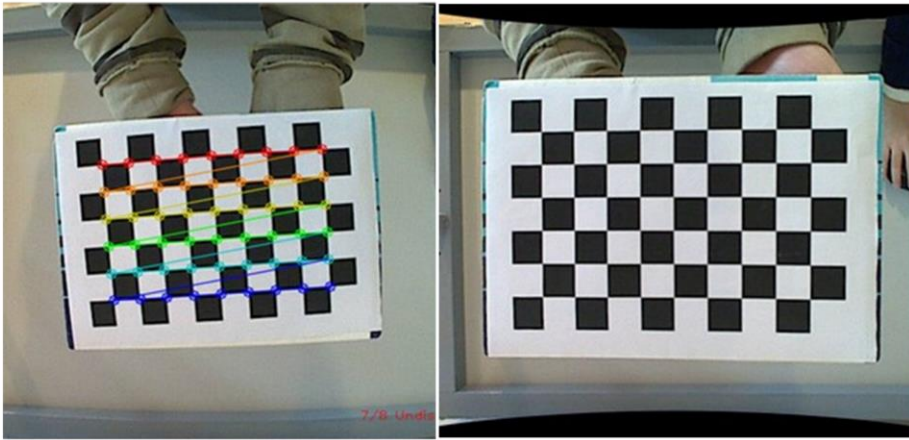
- $\mathbf{A}^{-}$ is a general inverse matrix
- if the number of control points is higher than the number of coefficients, pseudo-inverse matrix computation is used

$$\mathbf{A}^{-} = [\mathbf{A}^T \cdot \mathbf{A}]^{-1} \mathbf{A}$$

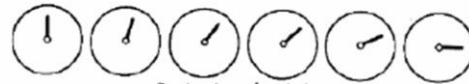
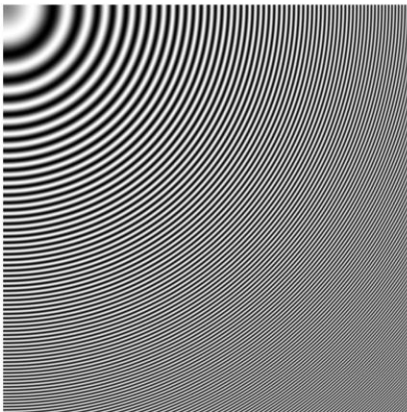
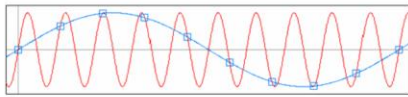
If we don't know the display/lens distortion coefficients, we can estimate the coefficients using error-minimization method.

Have control point pairs with a known position (x, y) and corresponding points with distorted position (x', y') .

Equation for the error estimation is $E = \dots$, where vectors $\mathbf{x}'$ and $\mathbf{y}'$ represents the coordinates of control points.



Example of camera calibration with image pattern with easily detectable corners of known positions [OpenCV].



Správné vzorkování



Aliasing při nižší rychlosti



Alias: ručička skáče z jedné pozice na druhou



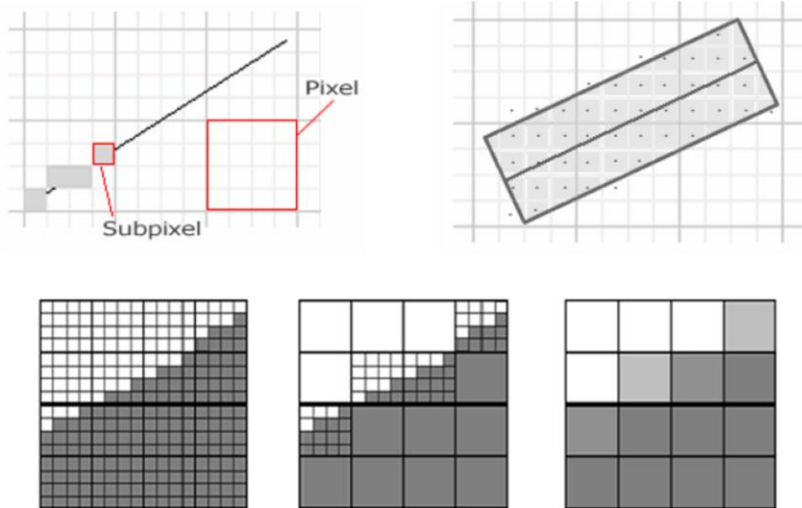
Alias: ručička se otáčí opačně



Bliká nebo roztává statická

Alias - a phenomenon that arises from **sampling the signal of a relatively high frequency with low frequency**.

- Alias when displayed (sampling) textures.
- Time zone alias.
- Arises in geometrical transformations.

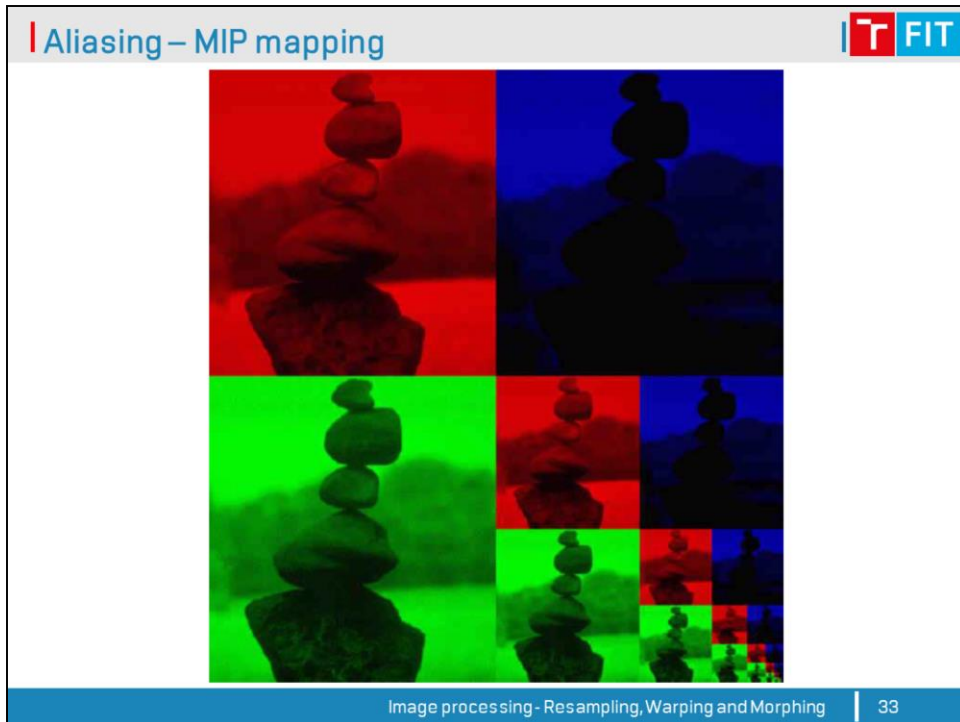


Supersampling - sampling with higher frequency.

- Insert more than one sample into the pixel – an average color/intensity.
- According to the pattern placement in the pixel we divide into
 - Regular supersampling
 - Stochastic / random supersampling

More computational power is needed.

Multisampling - supersampling only at edges objects.



MIP mapping – MIP is from „multum in parvo“ (a great deal in a small space).

- Multiple images with different resolution is stored in one image.
- Increase display speed.
- Eliminate alias formation when displaying large textures from large distance.
- Texture preprocessing - sampling for different resolutions with advanced value interpolation.
- Deciding which resolution to use according to distance of a texture from a camera.

- M. Sonka, V. Hlavac, R. Boyle. Image Processing, Analysis and Machine Vision (3rd edition). Thomson Learning. 2007
- Pratt, W., K.: Digital Image Processing: PIKS Inside, Third Edition. John Wiley & Sons, Inc. 2001.
- Zara, J. et al. Moderní počítačová grafika. Computer Press, 2010.

Conclusion

- Without geometric transformations it is not modern vector computer graphics and its hardware acceleration!
- Optimization of geometric transformations using homogeneous coordinates.
- The problem of reconstruction of a continuous signal from its discrete representation - values between samples not defined.
- Risk of aliasing when applying geometric transformation on a discrete image.